order functions, and Java Time APIs. We can get these features by targeting higher versions of Android, which is not a feasible solution. This problem can also be solved using third party libraries like RetroLambda, RxJava, etc, which is acceptable.

- There are also restrictions to adding functionality to Android APIs. I'm sure you might have felt like adding a particular feature such as being able to check if a string is an email or not, but you were restricted from doing so. Due to such restrictions, we end up making lots of 'Util' classes to solve the problem.
- Moreover, Android loves inheritance; it requires extending things like Activity, etc, which is not totally bad but limits flexibility.
- In Android, we need to write a lot of boilerplate code to achieve something. For example, if you want to write a model POJO class, you end up writing 40-50 lines of code, depending upon the number of attributes you add.
- In Java, all the empty values are set as null for efficiency. Android tries to be efficient by setting 'null' as the default value everywhere, which again comes with NPEs and more instances of the app crashing.
- And there's a lot more.

Some big players using Kotlin in production

JetBrains is already using Kotlin to build its IDEs. Its new IDE Rider is completely written in Kotlin while new features in older IDEs are also written in it. Pinterest, Coursera, Atlassian, Evernote, Basecamp and many more players are using Kotlin in their Android apps, either partially or fully. Gradle is introducing Kotlin as a language for writing build scripts, instead of Groovy.

Syntax of Kotlin

Let's take a crash course in the basic syntax of Kotlin. We won't cover all of it; hence, I recommend going to the Kotlin website and reading the documentation, which is pretty good, and also trying out the Kotlin Koans — they are very interesting and help you learn faster.

Declaring variables

Declaring variables in Kotlin is simple; you either write 'val' or 'var' and the name of the variable. Then the type of the variable which is optional provided that the compiler is able to figure it out.

```
// String (Implicitly inferred)
val spell = "Lumos"

// Int
val number = 74
```

The type can be explicitly specified after the variable name with a colon (:).

```
// Explicit type
var spells: List<String> = ArrayList()
```

Did you notice there is no 'new' keyword? That is because there is no 'new' keyword in Kotlin.

As we discussed earlier, there are two ways to declare variables in Kotlin:

1. val
2. var

In Kotlin, immutable values are preferred whenever possible. The fact that most parts of our program are immutable provides lots of benefits, such as a more predictable behaviour and thread safety.

Functions

```
fun add(a: Int, b: Int): Int {
   return a + b
}
```

This is how we write functions in Kotlin. The functions start with a 'fun' keyword (expecting the function to be fun to write for you), the parameters are reversed, and the return is typed at the end after the colon ':'. Pretty simple, right?

You can even cut the code of the function to this, as follows:

```
fun add(a: Int, b: Int) = a + b
```

And you can add default values to the function, as shown below:

```
fun add(a: Int = 0, b: Int = 0) = a + b
```

You also get a named argument! Did you notice that there is no semi-colon in the return statement? That's because Kotlin is a semi-colon free language.

Some amazing features of Kotlin

Let's look at what makes Kotlin so interesting.

Null safety

One of the common reasons for Android apps to crash is 'NullPointerException'. A lot of time is wasted debugging an issue and solving it. The great thing about Kotlin is that it comes with null safety, which helps in catching nulls at the compile time rather than during a runtime crash. The following code shows how this is done:

```
// Compile time error
val spell: String = null

// OK
val spell: String? = null
```